SIPGN

Choosing a Vector Database

Tradeoffs across pgvector, Pinecone, and Qdrant for RAG

Project Engineer Team

June 2026 1.0

About this document

COMPANY

SIPGN

AUTHOR

Project Engineer Team

EMAIL

harry.sitohang@inadigital.co.id

Contents

1

Why this guide exists

Retrieval-augmented generation (RAG) lives or dies by its retrieval layer. The vector database is where your documents become searchable by *meaning* rather than by keyword, and the choice you make here shapes operational cost, latency, and how much infrastructure your team has to babysit.

This guide compares three popular options — **pgvector**, **Pinecone**, and **Qdrant** — and shows how a RAG application actually talks to whichever store you pick. The mechanics are the same across all three; the tradeoffs are in where the index runs, who operates it, and how it scales.

In Plain English

In Plain English. A vector database stores lists of numbers (embeddings) that capture the meaning of a chunk of text. When a user asks a question, you turn the question into the same kind of list and ask the database "which stored chunks point in roughly the same direction?" The closest matches are your context. That is all "semantic search" means here.

2

How a RAG app talks to the vector store

There are two paths, and keeping them separate is the key mental model.

The **offline indexing path** runs at build time: you load documents, split them into chunks, embed each chunk into a vector, and *upsert* those vectors into the store. The **online query path** runs per request: the app embeds the user's question, asks the store for the top-k nearest vectors, stuffs the returned chunks into a prompt, and calls the LLM for a grounded answer.

The vector store only ever does two things in this picture: it *accepts upserts* during indexing and it *answers approximate-nearest-neighbour (ANN) searches* during queries. Everything else — chunking, embedding, prompt assembly — is your application's job. That is why migrating between stores is usually mechanical: the contract is small.

Pro Tip

Keep your embedding model and your vector store as separate, swappable components. Embeddings are just arrays of floats with a fixed dimension; any of the three stores in this guide can hold them. Pinning your retrieval code to a clean `embed()` / `upsert()` / `search()` interface means you can re-benchmark a different store later without rewriting the app.

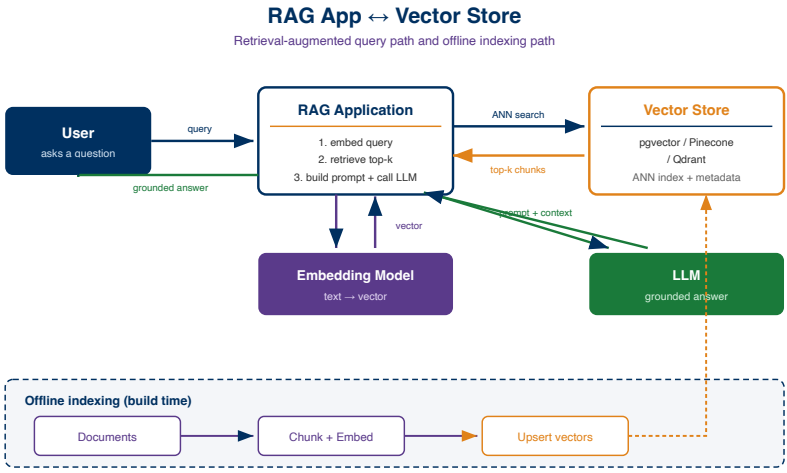

Figure 2.1: How a RAG application talks to the vector store (query path and offline indexing path)

3

The three contenders

3.1 pgvector

pgvector is a PostgreSQL extension that adds a `vector` column type plus ANN indexes (IVFFlat and HNSW). Its great virtue is that it is *just Postgres* — you keep your relational data, your transactions, and your existing backups, and bolt vector search on top. If you already run Postgres, adding pgvector can be a single `CREATE EXTENSION` away.

The tradeoff is that very large indexes and very high query volumes push Postgres harder than a purpose-built engine, and you own the tuning (index lists, `ef_search`, memory) yourself.

3.2 Pinecone

Pinecone is a fully managed, serverless vector database. You never run a node: you create an index, upsert vectors over an API, and query. It handles sharding, replication, and scaling for you, which is ideal for teams that do not want to operate infrastructure.

The tradeoff is that it is a proprietary hosted service — your vectors live in someone else's cloud, pricing is usage-based, and you have less control over the internals than with a self-hosted engine.

3.3 Qdrant

Qdrant is an open-source vector database written in Rust that you can self-host or run as managed cloud. It offers rich payload filtering, HNSW

indexing, quantization for memory savings, and strong performance, while keeping the option to run entirely on your own hardware.

The tradeoff is that, self-hosted, you operate it — deployment, upgrades, and capacity planning are yours, though the managed cloud tier removes most of that.

4

Comparison at a glance

Dimension	pgvector	Pinecone	Qdrant
Deployment model	Postgres extension (self-host)	Fully managed / serverless	Open-source self-host or managed
Operations burden	You run Postgres + tuning	Near-zero (vendor runs it)	You run it (or use managed tier)
Data locality	Your database / your cloud	Vendor cloud	Your hardware or vendor cloud
Indexing	IVFFlat, HNSW	Proprietary, managed	HNSW + quantization
Metadata filtering	Full SQL `WHERE` joins	Metadata filters	Rich payload filtering
Cost shape	Existing DB cost + compute	Usage-based subscription	Infra cost (or managed plan)
Best fit	Already on Postgres, modest scale	Want zero-ops, fast to ship	Want control + scale, open-source

Note

There is no universally "best" store here. The honest selection rule is: if you already run Postgres and your scale is modest, start with pgvector; if you want to ship fast with no infrastructure, reach for

Pinecone; if you want open-source control and room to scale, choose Qdrant. Re-benchmark when your corpus or query volume grows by an order of magnitude.

Watch Out

Embedding dimension and distance metric (cosine vs. dot vs. Euclidean) must match between how you embed and how the index is configured. A mismatch does not error loudly — it silently returns poor matches, which is far harder to debug than a crash.

5

A decision shortcut

> *Pick the store that minimises the operations your team will regret in six months — not the one that benchmarks fastest on a slide today.*

For most teams the decision collapses to a single question: *who do you want to operate the index?* If the answer is "nobody, just give me an API," Pinecone wins. If it is "we already run a database and want one less moving part," pgvector wins. If it is "we want full control and open-source, and we can run a service," Qdrant wins.

6

Glossary

Term	Meaning
RAG	Retrieval-augmented generation — feeding retrieved context to an LLM so its answer is grounded in your data.
Embedding	A fixed-length array of floats that represents the meaning of a piece of text.
Vector store	A database that indexes embeddings and answers nearest-neighbour searches over them.
ANN	Approximate nearest neighbour — finding the closest vectors quickly without scanning every one exactly.
Upsert	Insert-or-update: writing vectors (and their metadata) into the store during indexing.
Top-k	The k closest matches returned for a query, used as context for the LLM.
HNSW	Hierarchical Navigable Small World — a graph-based ANN index used by pgvector and Qdrant.
IVFFlat	An inverted-file ANN index available in pgvector that partitions vectors into lists.

Term	Meaning
Payload / metadata	Non-vector fields stored alongside an embedding, used to filter search results.
Quantization	Compressing vectors to use less memory, trading a little accuracy for capacity.